

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent	12386949
Kind Code	B2
Date of Patent	August 12, 2025
Inventor(s)	Ngiam; Jiquan et al.

Protection of user credentials during code execution

Abstract

Techniques are presented for managing access to external resources while executing code. The method involves receiving a request to execute code and sending an identifier (ID) for the code execution and credentials for accessing an external resource to a trusted server. The request is sent to execute the code with the ID to a code executor, and the code is executed. During code execution, an operation for accessing the external resource is detected. In response to the detection, a request to access the external resource is sent from the code executor to the trusted server. The trusted server accesses the external resource using the provided credentials and sends a response with the results back to the code executor. The code executor does not have access to the credentials for accessing the external resource. After the code terminates execution, the results from the code execution are presented on a device.

Inventors: Ngiam; Jiquan (Los Altos, CA), See; Bo Zhi (Redwood City, CA), Vasudevan; Vijay (Los Altos Hills, CA), Newman; Joshua (Mountain View, CA), Charumilind; Jaran (Palo Alto, CA), Rasi; Marc (Sunnyvale, CA)

Applicant: Dependable AI, Inc. (Los Altos, CA)

Family ID: 95156532

Assignee: Dependable AI, Inc. (Los Altos, CA)

Appl. No.: 18/375804

Filed: October 02, 2023

Prior Publication Data

Document Identifier	Publication Date
US 20250111036 A1	Apr. 03, 2025

Publication Classification

Int. Cl.: G06F21/53 (20130101); G06F21/44 (20130101); H04L9/40 (20220101)

U.S. Cl.:

CPC G06F21/53 (20130101); G06F21/44 (20130101); H04L63/083 (20130101); H04L63/10 (20130101); G06F2221/033 (20130101)

Field of Classification Search

CPC: G06F (21/53); G06F (21/44); G06F (2221/033); H04L (63/083); H04L (63/10)

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
10007790	12/2017	Biswas	N/A	G06F 21/31
11546324	12/2022	Brooker	N/A	H04L 63/20
2016/0095057	12/2015	Guy	709/224	H04L 67/1008
2017/0061566	12/2016	Min	N/A	H04L 47/822
2018/0109504	12/2017	Poffenbarger	N/A	H04L 9/3234
2022/0263891	12/2021	Bernat	N/A	H04L 67/1008
2024/0103942	12/2023	Jasti	N/A	G06F 9/542

Primary Examiner: Le; Chau

Attorney, Agent or Firm: Schwegman Lundberg & Woessner, P.A.

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATIONS

(1) This application is related by subject matter to U.S. patent application Ser. No. 18/228,216, entitled “Utility System for Automated Code Generation and Execution,” filed on Jul. 31, 2023, which is incorporated by reference in its entirety.

TECHNICAL FIELD

(2) The subject matter disclosed herein generally relates to methods, systems, and machine-readable storage media for a utility system for the automated generation and execution of computer programming code.

BACKGROUND

(3) Software engineering gives us the ability to create and automate. However, it has traditionally been expensive; thus, many problems and tasks at work and in people's lives have not been automated. Foundation models (FMs) are changing this equation, and it is now possible for FMs to obtain working code from requests expressed in natural language. However, FMs still need an environment to operate in (e.g., need to run pip install, copy-paste the code into a script, and work with a terminal), and there is still the need for an engineer to supervise and make sure all the pieces work together. Therefore, the benefits of FMs are not within the reach of everyone.

(4) Developers may want to share their code with other users; however, other users may not trust the code shared by other users because it may contain software that may access private data for private resources, e.g., email accounts, files on a server. Some sharing solutions utilize a sandbox

with limited access (e.g., no Internet access). Still, these sandboxes are designed to lock down access fully, which makes it impossible to perform certain operations, such as sending an email for the user. Other solutions require that the user fully trust their application developer, such as applications that execute on a mobile device.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

- (1) Various appended drawings illustrate example embodiments of the present disclosure and cannot be considered as limiting its scope.
- (2) FIG. 1 is a user interface (UI) for entering a request as a prompt to generate a plan, according to some example embodiments.
- (3) FIG. 2 is a UI showing the generated plan according to some example embodiments.
- (4) FIG. 3 is a flowchart of a method for generating a plan and executing the operations within the plan, according to some example embodiments.
- (5) FIG. 4 illustrates the process for executing the plan's operations according to some example embodiments.
- (6) FIG. 5 is a flowchart of a method for generating and executing code for the plan, according to some example embodiments.
- (7) FIG. 6 is a UI showing the operations being executed according to some example embodiments.
- (8) FIG. 7 is a UI showing the results after executing the plan, according to some example embodiments.
- (9) FIG. 8 illustrates two versatile system-provided functions according to some example embodiments.
- (10) FIG. 9 illustrates the execution of actions by the Limited Python (LP) Executer that is part of the Large Language Model (LLM) Code Interpreter and Execution Environment (LCIEE), according to some example embodiments.
- (11) FIG. 10 is a sample architecture for the LCIEE, according to some example embodiments.
- (12) FIG. 11 is a flowchart of a method for processing a request by the LCIEE, according to some example embodiments.
- (13) FIG. 12 illustrates the generation of a UI utilizing LCIEE, according to some example embodiments.
- (14) FIG. 13 is a flowchart of a method for utilizing the task marketplace, according to some example embodiments.
- (15) FIG. 14 is a sample UI for accessing the task marketplace, according to some example embodiments.
- (16) FIG. 15 illustrates the option to generate additional code during the execution of the code and then executing the additional code according to some example embodiments.
- (17) FIG. 16 is another example of the generation of the plan, according to some example embodiments.
- (18) FIG. 17 shows a document generated by the plan, according to some example embodiments.
- (19) FIG. 18 details the implementation plan according to some example embodiments.
- (20) FIG. 19 is a flowchart of a method that uses recursiveness to expand operations into a plurality of sub-operations, according to some example embodiments.
- (21) FIG. 20 is a flowchart of a method for implementing a tool that generates and executes the code in a controlled environment to satisfy user requests entered as text prompts, according to some example embodiments.
- (22) FIG. 21 illustrates creating a task for the marketplace, according to some example embodiments.

- (23) FIG. 22 illustrates the creation of a task that combines multiple tasks, according to some example embodiments.
- (24) FIG. 23 details the LCIEE's architecture for managing access to external resources, according to some example embodiments.
- (25) FIG. 24 shows the flow of operations to access external resources while executing code, according to some example embodiments.
- (26) FIG. 25 is a flowchart of a method for managing access to external resources while executing code in a safe environment, according to some example embodiments.
- (27) FIG. 26 is a block diagram illustrating an example of a machine upon or by which one or more example process embodiments described herein may be implemented or controlled.

DETAILED DESCRIPTION

- (28) Example methods, systems, and computer programs are directed to a tool that generates and executes the code in a controlled environment to satisfy user requests entered as text prompts. Examples merely typify possible variations. Unless explicitly stated otherwise, components and functions are optional and may be combined or subdivided, and operations may vary in sequence or be combined or subdivided. In the following description, numerous specific details are set forth to provide a thorough understanding of example embodiments. It will be evident to one skilled in the art, however, that the present subject matter may be practiced without these specific details.
- (29) In one aspect, an environment to use FMs for coding and execution of the programming code is provided, where the code produced by the FMs is operated safely and reliably. In this environment, users can create tools to automate tasks in their lives using simple natural language, and these tools serve as the glue between the other utilities that users access in their work and lives.
- (30) In one aspect, FMs are used to translate requests entered as text into a plan and code that are executed by the service. The code is executed in a custom Large Language Model (LLM) Code Interpreter and Execution Environment (LCIEE) that controls the functions that may be executed, the errors that may occur, and accesses Application Programming Interfaces (APIs) to access external servers for data. LCIEE features are bundled into a cohesive service that enhances LLM's usability, versatility, and safety in coding applications, thus bringing the power of LLMs to the reach of everyone; no software engineer is required to design and execute programs.
- (31) The present techniques separate the execution of code from the connectors requiring credentials to access external resources. In particular, a task includes (a) the executable code performing the useful aspects of the task and (b) the credentials enabling certain actions to access external resources (e.g., read external data, write to a file stored in a cloud environment).
- (32) When executing the code in a safe sandbox, calls for external access are routed to a separate trusted server. During the setup of the sandbox and the trusted server, each sandbox is assigned a unique identifier to identify itself and be validated by the trusted server. The trusted server manages the credentials for external resources and performs the operations associated with the external access (e.g., returns requested data to the code executing in the sandbox).
- (33) Using the trusted server allows the packaging of tasks, stored in a marketplace of shareable tasks, as packages of code and credentials. If a new task includes access to multiple tasks (each task may include its own credentials), the new task will include calls to the multiple segments of code from the multiple tasks and the set of credentials included in the multiple tasks. This architecture achieves the goal of allowing users to share tasks while retaining a high degree of trust that their data and their credentials remain secure.
- (34) Techniques are presented for managing access to external resources while executing code. The method involves receiving a request to execute code and sending an identifier (ID) for the code execution and credentials for accessing an external resource to a trusted server. The request is sent to execute the code with the ID to a code executor, and the code is executed. During code execution, an operation for accessing the external resource is detected. In response to the detection, a request to access the external resource is sent from the code executor to the trusted server. The

trusted server accesses the external resource using the provided credentials and sends a response with the results back to the code executor. The code executor does not have access to the credentials for accessing the external resource. After the code terminates execution, the results from the code execution are presented on a device.

(35) FIG. 1 is a user interface (UI) **102** for entering a request as a prompt to generate a plan to perform tasks required to satisfy the request, according to some example embodiments. The UI **102** is presented when the user selects the option to create a new plan.

(36) The user may enter a prompt in the prompt field **104** with a request entered as natural language. After the user completes the request in the prompt field **104**, the user can select option **106** to generate the plan.

(37) In the illustrated example, the user has entered the following prompt: Research information about Brittany Spaniels and using that info: 1. Write a haiku about Brittany Spaniels and post it to twitter and facebook. 2. Download an image and post it to snapchat, instagram, tiktok, and pinterest. 3. Draft an email with a summary of the info you find.

(38) Thus, the user has requested a poem about a breed of dog, Brittany Spaniels, and to post the poem on Twitter and Facebook. Further, the user has asked to download an image of the dog and post it to Snapchat, Instagram, TikTok, and Pinterest. Finally, the user has requested to draft an email with a summary of the information obtained.

(39) A second example of a prompt would be, “Find the top articles on news.ycombinator.com and summarize them. Just read the news site directly to find the top articles.” Thus, the user is requesting that LCIEE access a website to retrieve top articles and then provide a summary, and specifically, reading from the news site directly.

(40) It is noted that Generative AI tools like ChatGPT provide responses based on the data used for their training, and these tools do not have the capability to perform searches or access websites to obtain information. However, LCIEE generates code that can perform this type of operation to access the Internet to get information and then process it using LLMs. Therefore, the information generated by LCIEE covers up to the most recent information available, unlike ChatGPT, that only covers information in the data used to train the model. The LCIEE also permits code to retrieve and push data to other applications, allowing users to gather and process information from their existing applications.

(41) To better understand the embodiments described below, a few terms used in the Specification are now defined: A USER PROMPT is text entered via the user interface that specifies a request from the user. Unless otherwise specified, a user prompt may be referred to simply as a prompt. An LLM PROMPT is text to be used as input for an LLM. A PLAN contains a sequence of operations to be performed in order to perform actions in the request received via the prompt. The operations can be expanded recursively to generate a list of sub-operations to be performed. The plan contains a natural-language description, also referred to as human-readable text. CODE refers to programming instructions, to be executed by one or more processors, to execute the plan; that is, the plan is converted to code that represents the operations in the plan. A COMMENT is a textual description embedded in the code that explains what the code is doing; the comment is not executed by the processor as a computer instruction. The code may execute ACTIONS and FUNCTIONS. A FUNCTION refers to a block of code that performs a specific task. A function typically takes some input values called arguments or parameters, performs a set of operations, and optionally returns a result. Functions can be of many types and may be used, for example, to calculate a value, format data, summarize data, store data, etc. An ACTION is programming code that interacts with another computer (e.g., with the “outside world”). For example, an action may perform a function to get data from an external source via an Application Programming Interface (API). During execution of the plan, the UI may show the current action or function being executed. Also, during execution, the UI may show the comment, associated with the current action or function, which explains what the code is doing at that moment. A completed plan may be saved

as a TASK that can be reused in the future. The task is a type of plan that has already been executed or otherwise validated. The task may be saved in a TASK MARKETPLACE that stores the current list of tasks available to the user. A SAVED TASK may be used as part of a plan in the future.

(42) FIG. 2 is a UI **202** showing the generated plan according to some example embodiments. The UI **202** shows, in plan field **204**, the plan generated by an LLM to satisfy the request described with reference to FIG. 1.

(43) The generated plan includes the following operations: 1. Search the internet for Information about Brittany Spaniels. 2. Extract the useful details from what was found. 3. Write a short poem about Brittany Spaniels based on the details. 4. Share the poem on Twitter and Facebook. 5. Search for photos of Brittany Spaniels and download one. 6. Post the photo on Snapchat, Instagram, TikTok and Pinterest. 7. Draft an email summarizing the information found about Brittany Spaniels.

(44) LCIEE utilizes the plan to generate code to execute each operation or to expand one operation to generate a list of sub-operations to be performed. That is, LCIEE code generation is recursive, where the sub-operations may be used to generate code or to generate a new list of sub-operations. This way, complex plans may be gradually broken down into simpler operations that can be performed by code.

(45) Some operations require accessing the Internet, such as performing a search or posting an item on an online service. Other operations process data and LLM's may be used to perform this processing, such as extracting details from found information, writing a poem, or drafting an email summarizing the information.

(46) LCIEE checks the request in the prompt and determines if there are ambiguities in the request, such as a lack of information to perform the request. In this example, ambiguity field **206** has flagged that the email address for the email is missing. If the user clicks on the ambiguity field **206**, another UI may be presented to enter the missing information. In other embodiments, the user may return to the UI **102** and redo the prompt by adding the missing information.

(47) LCIEE also checks the tools that are needed to complete the plan. In this case, access to the different online services has to be provided, as presented in tools field **208**. The user may select each tool and provide credentials information to access the tool.

(48) Regarding the second example, the plan generated by LCIEE is the following: 1. Go to a popular technology news website and find the most popular articles. 2. For each of the top articles, get the full content from the website. 3. Summarize each article into a short summary. 4. Extract the title, web address, and summary for each article. 5. Combine all the summaries into one long summary with each summary on its own line. 6. Use the tool that can generate coherent writing to combine all the individual summaries into one high-level overview. 7. Provide the high-level overview.

(49) FIG. 3 is a flowchart of a method **300** for generating a plan and executing the operations within the plan, according to some example embodiments. While the various operations in this flowchart are presented and described sequentially, one of ordinary skill will appreciate that some or all of the operations may be executed in a different order, be combined or omitted, or be executed in parallel.

(50) At operation **302**, the prompt is received via the UI, such as the UI **102** from FIG. 1. From operation **302**, the method **300** flows to operation **304** to create the plan with the list of operations. In some example embodiments, the plan is generated using a Foundation Model (FM). The FM is an LLM on a vast quantity of data at scale (often by self-supervised learning or semi-supervised learning), such that it can be adapted to a wide range of downstream tasks. Some example FMs include Google's PaLM, Anthropic's Claude, and various early GPT foundation models, e.g., OpenAI's GPT-n series (e.g., GPT-4).

(51) For example, the prompt used for ChatGPT may include a preamble provided by the system (e.g., "generate a plan for") followed by the user-entered prompt. More complex examples of LLM

prompts are provided below with reference to FIG. 5 and others.

(52) From operation **304**, the method **300** flows to operation **306** to generate the code using the LLM. For example, the LLM prompt may be, “please create a Python program to performing the following plan,” plus the plan previously created.

(53) In other embodiments, operations **304** is optional, and the code is generated directly using an LLM based on the prompt entered by the user. Then, the operations in the plan are extracted from the code (e.g., the comments describing each of the first-level function calls).

(54) Some embodiments are described with reference to the creation and execution of Python scripts, but the same principles may be used for other programming languages, such as JavaScript, Java, C++, etc. The embodiments presented should therefore not be interpreted to be exclusive or limiting, but rather illustrative.

(55) From operation **306**, the method flows to operation **308**, performed for all the operations in the plan, to perform the operation by executing the corresponding code. It is noted that the embodiment described in FIG. 3 is for a plan that generates directly-executable operations, also referred to herein as atomic operations, which are operations that can be executed by a function call in the code. Other embodiments may include operations that have to be broken down into smaller suboperations, i.e., the operation acts like a mini-plan that needs to be expanded. The recursion of operations is described in more detail below with reference to FIG. 20.

(56) However, running directly the code generated by the LLM may easily run into trouble. People using LLMs have found that LLMs may generate good code because there is a large corpus of code to train the LLMs, but that the code does not always run at the first try. There may be logical mistakes, calls to functions that do not exist in the libraries identified for the code, mismatch of arguments (e.g., passing an integer when a real number is required), access to private information, access to unauthorized servers, etc. This is why LCIEE provides a controlled environment to execute the code and manage execution errors.

(57) It is noted that most users do not have access to the code that is being executed, unless the user has management privileges to run in debug mode and see the code when the execution is in progress.

(58) The generated code is able to perform functions that may access an external resource, such as by using an API to access a server and get data from a database in the server or requesting a search from a search engine.

(59) In some embodiments, LCIEE includes a Limited Python (LP) executor that is an interpreter for Python code but includes additional limitations, such as controlling how errors are handled or disallowing certain function calls (e.g., do not allow LCIEE to send an email for the user without an express request from the user). Thus, in some example embodiments, operation **308** is executed by the LP executor.

(60) The LP executor analyzes each line of code, determines if the code is safe to execute and if access to a resource is required, determines if there is authorization to access the resource.

(61) For example, in Python there is a function called `eval()`, which is a versatile function that evaluates a string expression as a Python expression and returns the result. The LP executor removes the ability to call `eval`, which prevents unsafe expressions from being executed. This ability to precisely enable and disable certain function execution is further applied to actions that interact with user's applications. For example, the LP executor may provide access to read the user's email but may not provide access to send emails for the user.

(62) If the LP executor finds an error during execution (operation **310**), then the method flows to operation **312** to determine if the execution should quit or not. If there is no error, then the method **300** flows to operation **320**.

(63) When execution runs into an error, the LCIEE may decide to redo the code at operation **314** and then try to execute the operation again. This means asking the LLM to regenerate the code for that operation. For example, the prompt for the LLM includes information about the code being

executed, the error generated, and the operation, and then request to generate new code to try again.

(64) The process may repeat until the regenerated code is executed without errors or until a predefined number of iterations are performed. For example, the LP executer may decide to quit after trying to run the code three times, but other number of iterations may be utilized.

(65) If LCIEE decides to quit at operation **312**, the method **300** flows to operation **316** to terminate execution of the plan and notify the user.

(66) At operation **320**, a check is made to determine if this is the last operation to be executed. If this is not the last operation, operation **318** is executed to provide feedback with the results from this task for the plan, and then the next task is executed. If this was the last operation, at operation **322**, the results are presented on the UI of the user.

(67) Below is an example of the Python code generated for the example described above to summarize the news:

```
(68) TABLE-US-00001 # Function to find and summarize top articles on news.ycombinator.com
def summarize_top_hacker_news_articles( ):    # 1. Go to news.ycombinator.com and find the top
articles    hacker_news_url = "https://news.ycombinator.com"    hacker_news_content =
get_website_content(hacker_news_url)    if not hacker_news_content.success:    return 'Could
not get contents of ' + hacker_news_url + '\n' + 'Error:' + hacker_news_content.error_message    #
2. Use data_extractor to extract the top article titles and URLs.    top_articles =
data_extractor("Extract top article titles and URLs.", hacker_news_content.value. """"({ "title"; "...",
"url": "..."}""")    if not top_articles.success:    return 'Could not extract top articles from ' +
hacker_news_url + '\n' + 'Error: ' + top_articles.error_message    # 3. For each article, use
get_website_content to get the full content and use universal writer to summarize each article.
summaries = [ ]    for article in top_articles.values:    article_content =
get_website_content(article["url"])    if article_content.success:    summary =
universal_writer("summarize this article for a general audience", article_content.value, "simple text
string")    if summary.success:    summaries.append({"title": article("title"), "url":
article("url"), "summary": summary.value})    else:    summaries.append({"title":
article("title"), "url": article["url"], "summary": "Could not summarize this article."})    else:
summaries.append({"title": article["title"], "url": article["url"], "summary": "Could not get
article content."})    # 4. Join the summaries into a single string with newlines.
joined_summaries = "\n".join([f'{s["title"]} ({s["url"]})\n{s["summary"]}\n' for s in summaries])
# 5. Use universal_writer again to combine the summaries into a high-level overview.    overview
= universal_writer("create a high-level overview of the top articles on Hacker News",
joined_summaries, "simple text string")    if overview.success:    return overview.value    else:
return 'Could not create a high-level overview of the top articles on Hacker News.\n' + 'Error:
' + overview.error_message output = summarize_top_hacker_news_articles( ) output
```

(69) The code includes functions such as `get_website_content( )` that accesses a website based on the URL provided as an argument, `data_extractor( )` which extracts the top article titles and URLs, and `universal_writer( )`, which is described in more detail below with reference to FIG. 9.

(70) Besides finding errors during execution, the execution of the plan may run into other problems, such as being unable to finish because the execution was derailed (e.g., an infinite loop). The LP executer includes other measures to avoid these problems, such as limiting the number of times a for loop is executed (e.g., quit execution after running the for loop ten times) or providing a maximum time of execution based on the complexity of the plan.

(71) Another measure may include assigning an execution budget to the plan, such as the execution budget being decreased as the user executes code (e.g., function calls incur a cost against the budget). Once the budget is exhausted, the LP executer terminates or suspends execution of the plan.

(72) Here is another example of the code generated for a prompt that requested “Analyze big tech companies and store information in a Google Sheet.”

```

(73) TABLE-US-00002 # Analyze big tech companies and store information in a Google Sheet. def
analyze_big_tech_companies(): # Define a list of big tech companies to analyze. companies =
["Apple", "Microsoft", "Amazon", "Google", "Facebook"] # Initialize a list to store the
company information and news summaries. company_data = [] # Iterate through each
company in the list. for company in companies: # Obtain the company's stock ticker
symbol. ticker_result = finance_get_ticker(company) if not ticker_result.success:
continue ticker = ticker_result.value # Get the stock price and market cap.
quote_result = finance_get_quote(ticker) if not quote_result.success: continue
quote = quote_result.value # Get the latest news related to the company. news_result =
finance_get_news_for_symbol(ticker) if not news_result.success: continue
news_data = news_result.value # Find additional news articles if needed.
additional_news_result = news_search(company) if additional_news_result.s1
news_data.extend(additional_news_result.value) # Summarize the news articles.
news_summaries = [] for news in news_data: if news.url: content_result =
get_website_content(news.url) if content_result.success: summary_result =
universal_writer("summarize this news article", content_result.value, "simple text string")
if summary_result.success:
news_summaries.append((summary_result.value, news.url)) # Store the company information
and news summaries. company_data.append((company, ticker, quote.regular_market_price,
quote.market_cap, news_summaries)) # Create a new Google Sheet. sheet_result =
google_file_create("Big Tech Companies Analysis", "application/vnd.google-apps.spreadsheet")
if not sheet_result.success: return "Error creating Google Sheet: " +
sheet_result.error_message sheet_id = sheet_result.value.id # Format the spreadsheet with
appropriate headers. headers = ["Company Name", "Stock Ticker", "Stock Price", "Market
Cap", "News Summary", "News URL"] google_sheets_append(sheet_id, "Sheet1!A1",
[headers]) # Add the company information and summarized news to the spreadsheet. for
company_info in company_data: company, ticker, price, market_cap, news_summaries =
company_info for summary, url in news_summaries: row_data = [company, ticker,
price, market_cap, summary, url] google_sheets_append(sheet_id, "Sheet1!A:A",
[row_data]) # Return the link to the created Google Sheet. return
f"https://docs.google.com/spreadsheets/d/{sheet_id}/edit" sheet_link =
analyze_big_tech_companies() sheet_link

```

(74) The result of this plan is a spreadsheet with information about tech companies Apple, Microsoft, Amazon, Google, and Facebook. The information includes items like the stock ticker, market cap, latest news, etc.

(75) In some example embodiments, the LCIEE generates an internal or intermediate plan that is not shown to users. This internal plan is more tied to the code than to the plan presented to users. The internal plan is included in the LLM prompt to be processed by the LLM to generate a natural-language plan for the user.

(76) For example, the following is an internal plan:

(77) TABLE-US-00003 1. Use the 'general_search' function to search for information about Brittany Spaniels. 2. From the search results, use the 'get_web_content' function to extract the content of the most relevant websites. 3. Use the 'universal_writer' function to summarize the information obtained from the websites. 4. Use the 'universal_writer' function again, this time to write a haiku about Brittany Spaniels using the summarized information as context. 5. Use the 'general_search' function again, but this time search for images of Brittany Spaniels. 6. From the image search results, select and download the most suitable image. 7. Use the 'gmail_create_draft' function to draft an email. The subject of the email will be "Information about Brittany Spaniels", the body will contain the summary of the information found, the haiku, and the downloaded image.

(78) FIG. 4 illustrates the process for executing the operations of the plan, according to some

example embodiments. In some prior art solutions, a plan **408** is executed in independent blocks of code, such that the entire history of the plan, execution, and outputs are repeatedly fed back to the LLM. For example, a first step **410** is to provide the plan, and execute a first program. Then, a second step **412** is executed, where the plan is provided, the output of program 1, and then a new block of code which is the program 2 that produces output 2. Each time a block is executed, the entire history is fed back to the LLM. This requires the LLM to be the execution engine that figures out relationships between the execution blocks. This quickly runs into context length limits with the LLM, and makes it challenging to maintain shared context (e.g., global variables, intermediate data obtained, etc.). It also quadratically increases the cost of execution since the entire history has to be continuously fed back to the LLM.

(79) The process is then repeated for different blocks of code N times, where the n .sup.th step **414** requires to be fed the outputs from all the previous programs that have executed to understand the context. That is, the more programs are executed, the faster the complexity to execute the plan grows.

(80) On the other hand, with LCIEE **402**, a single program **404** is executed, so the program can define program variables to store data that may be used by the different function calls and action calls. This makes possible to have larger programs and reduce complexity.

(81) Further, LCIEE allows the program to safely call actions that interact with other computers (“the outside world”). In prior art approaches, plugins are enabled (a plugin is a software component that adds a specific feature or functionality to an existing computer program). However, because other approaches only control the generation of code, but do not control the execution environment, it is very difficult to control the behavior of the program when executing plugins. On the other hand, LCIEE controls the execution environment, so even if a plugin misbehaves (i.e., does not work properly or executes malicious code), LCIEE will control what that the plugin does not execute forbidden instructions (e.g., root access, access to the OS data structures).

(82) FIG. 5 is a flowchart of a method **500** for generating and executing code for the plan, according to some example embodiments. While the various operations in this flowchart are presented and described sequentially, one of ordinary skill will appreciate that some or all of the operations may be executed in a different order, be combined or omitted, or be executed in parallel.

(83) Operation **502** is for receiving the user prompt, such as the one illustrated in FIG. 1. From operation **502**, the method **500** flows to operation **504** to generate a first LLM prompt that will be input for the LLM to generate a high-level plan. The first LLM prompt includes instructions to the LLM for generating the high-level plan and information about the prompt received from the user.

(84) In some example embodiments, the first LLM prompt includes fields that are filled in to complete the prompt. These fields include information based on multiple factors, such as the configuration for the user, the type of actions or external integrations needed, the final output format requested in the user prompt, etc. In the prompt examples presented herein, fields are presented as items enclosed within curly brackets (`{ }`). The curly brackets are not part of the final prompt and are shown as indicators or place holders for the data to be inserted.

(85) Here is a sample first LLM prompt to create a high-level plan:

(86) TABLE-US-00004 You are a detailed oriented person who likes to plan things. You come up with comprehensive high-level plans that are easy to follow and execute. They can involve loops and conditional statements. You may reference the following dataclasses:

`{DATACLASS_DEPENDENCIES}` You can use the following tools in your plans:

`{ACTIONS_DESCRIPTIONS}` You have access to a coding environment that supports loops, variables, conditionals. Do not use any tools not mentioned above. Assume that no documents exist until you create them. The following libraries have already been imported, and are available:

`{EXTERNAL_LIBS}` Important: - Plans should be a numbered list in markdown format. - Refer to the tools in a high-level way. After outputting the plan, put the names of the tools used, comma-separated, in `<Tools>` and `</Tools>` tags. Be comprehensive in listing which tools were used, even

if the high-level plan does not mention them explicitly by name. Example === Create a function that given a url, will: 1. First do a general_search to find relevant information from the web. 2. Then, extract all the urls from the search results. 3. Because the content on each site may have further links, we want to crawl them to find more urls and content. - Create another function to recursively to crawl the urls and extract the content. - Make sure the crawling is limited to a reasonable depth. - Use data_extractor to extract urls from website contents. - Use universal writer to summarize the content of each site, with attributions to the urls. - Use map_urls to map the short urls to full urls. 5. Save the summary to a document. 7. Return summary as the output. 8. Do the work with 'xet data founders' as the query. Call the function 'xet data founders' as the query.

<Tools>general_search, map_urls, data_extractor, universal_writer</Tools>

<Call>crawl_and_summarize('xet data founders')</Call> Please help create a thoughtful concise high-level plan for achieving this: {USER_PROMPT}

(87) For example, if the request involves accessing or creating a Google Sheets document, then the libraries will be added that support operations for Google Sheets. However, if the prompt is associated with a search for patent information (e.g., list prior art cited in prosecution of a certain patent application), then libraries for accessing patent resources will be included.

(88) From operation **504**, the method **500** flows to operation **506** where the first LLM prompt is fed as input to the LLM model, and the LLM model will return the high-level plan.

(89) From operation **506**, the method **500** flows to operation **508** to generate a second LLM prompt to generate the code that will be executed to implement the plan. The second LLM prompt utilizes the high-level plan plus additional information and fields. As with the first LLM prompt, the second LLM prompt utilizes fields based on the request and what the plan is aiming to achieve.

(90) Here is an example of the second LLM prompt:

(91) TABLE-US-00005 You are a creative, straight-forward and direct principal software engineer. You like to write code, and do not like to chat. You have access to a restricted Python interpreter. You can use loops, variables, conditionals, and basic python. Do not make any assumptions about the format of strings returned by functions, unless otherwise stated. You may reference the following dataclasses, but do not construct them directly: {DATACLASS_DEPENDENCIES} The following libraries have already been imported, and are available: {EXTERNAL_LIBS} You ***CANNOT*** import any other libraries, and are only limited to the following functions: {ACTION_DOC_STRINGS} Also, these functions are available: {BASE_OPS} You **MUST** produce comments along side the code so that a non- programmer can understand it extremely well. For each line of code, you must add a comment above it to express in a super simple way the intent of the code at that line. Each comment **MUST** be on one line. Requirements: - The output must have both a <Python> section and a <Call> section. - Only functions go in <Python></Python> tags. - The script to call the function with the task-specific arguments must go in <Call></Call> tags. - **DISALLOWED**: eval(), exec(), nonlocal, or global. You **MUST NEVER** use them. - Never import any libraries, assume all available libraries have already been imported. - Never create any dataclass or classes. - Handle errors by throwing exceptions. Use the most appropriate exception type depending on the error. - You can only use pydantic. create_model to create models. - You **MUST** define all your pydantic models at the **TOP** of the <Python> section. - You **MUST** use annotated assignments. Tips: - Try to write general functions instead of task-specific functions. - The arguments to the function in <Call></Call> can be specific to the task. - Include specifics of the goal in the context to universal writer verbatim. Be verbose. - Provide universal writer with more details in the task and context. - If tools document that they throw exceptions, try to handle them gracefully while still accomplishing the task. If an exception would prevent the task from being accomplished, then throw the exception. You may add additional context when throwing it. Here is the high-level plan: {HIGH-LEVEL PLAN}

(92) From operation **508**, the method **500** flows to operation **510** where the second LLM prompt is input for the LLM, which returns the code to be executed for the plan.

(93) From operation **510**, the method **500** flows to operation **512** where the code generated is executed by the LP executer (although some validations on the code may be performed before execution). At operation **514**, the results are presented on a UI.

(94) FIG. **6** is a UI **602** showing the operations that are being executed, according to some example embodiments. In some example embodiments, the operations are extracted from the comments of the code.

(95) In the illustrated example, the operations include the following:

(96) TABLE-US-00006 - Do a general_search for “Brittany Spaniels information” and extract relevant details. - Use universal writer to write a haiku about Brittany Spaniels based on the details. - Post the haiku to Twitter and Facebook. - Do an image search for “Brittany Spaniel” and download an image. - Post the image to Snapchat, Instagram, TikTok and Pinterest. - Use universal_writer to draft an email with a summary of the information found.

(97) Therefore, the user can see the progress being made during execution. If there is an error during the execution of the code, the error is presented in the UI **602**, with a selectable option to open another UI with more details on the error and the progress.

(98) FIG. **7** is a UI **702** showing the results after executing the plan, according to some example embodiments. The UI **702** shows the initial plan **704** and the results **706**. The results **706** for this example include the following: Post to Twitter: Brittany Spaniel is a hunting dog breed that is energetic and intelligent. They have floppy ears and a wagging tail, and they love to roam and bark. However, with early training and exercise, they can become the perfect family friend. Post to Facebook: Brittany Spaniel is a hunting the breed that is energetic and intelligent. They have floppy ears and a wagging tail, and they love to roam and bark. However, with early training and exercise, they can become the perfect family friend. Post image to Snapchat Post image to Instagram Post image to TikTok Post image to Pinterest Send email: Dear [Recipient], I hope this email finds you well. I wanted to share some information with you about Brittany Spaniels, a breed of dog that I recently researched. . . . Brittany Spaniels where originally bred in France for hunting purposes. They are medium-sized dogs with a muscular build and a distinctive orange and white coat. They are known for their high energy levels and of exercise, making them a great choice for active families. In terms of temperament, Brittany Spaniels are friendly and affectionate dogs that love to be around people. They are also highly intelligent and trainable, making them a good choice for first-time the owners. When it comes to care requirements, Brittany Spaniels need regular exercise and mental stimulation to keep them happy and healthy. They also require regular grooming to keep their coat in good condition. Overall, Brittany Spaniels make great pets for families who are looking for an active and affectionate companion. If you have any further questions about this breed, please don't hesitate to ask. Best regards, [Your Name]

(99) Thus, the results indicate performing the post to Twitter, the post to Facebook, the image posted to Snapchat, the posted image to Instagram, the posted image to TikTok, the posted image to Pinterest, and the email that was sent.

(100) FIG. **8** illustrates two versatile system-provided functions, according to some example embodiments. The two functions are universal_writer **802** and data_extractor **804**.

(101) The purpose of the function universal_writer is to perform any writing task by leveraging the power of LLMs. For example, when asked to produce a summary of “latest trends in AI,” one of the operations may be to “find a website covering the latest trends,” and then to “summarize it.” These operations may be autogenerated by the LLM. Other tasks may include “extract URLs,” “translate to English,” etc.; virtually any task related to the content could be included.

(102) The inputs for the universal_writer are a task to be performed (text format) and a context that defines the problem area. For example, the context may be a document, a website, a theme (e.g., “patents,” “news,” “dating,” etc). The output will depend on the requested task, e.g., a text summary, a list of items.

(103) It is interesting to note that the task to be used as input is generated automatically using the

LLM, not a human. The LLM figures out what is the right task to be performed and uses that as the input for the universal_writer.

(104) Further, inside the universal_writer, there may be the creation of an LLM prompt to generate code to perform the task requested. Additionally, the LLM prompt used to create the universal_writer may vary based on the operations of the plan. LCIEE will select the proper LLM prompt to be used to create the universal_writer.

(105) Here is an example of an LLM prompt used to generate the universal_writer function. This is an ingenious design to improve instructions. As before, fields in the LLM prompt example are enclosed in curved brackets, and the fields are filled in based on the request.

(106) TABLE-US-00007 You are a creative, straight-forward and direct instruction master. You like to just get to the point and get the job done. The model is able to take an instruction and then return a response. Using this ability, you MUST help your team write the best instruction for an LLM such as GPT3. You MUST be concise, clear and specific. NEVER suggest using any tools. NEVER come up with any names, facts, or information unless provided. NEVER produce any placeholders. You NEVER ask questions, or follow ups. Assume the task is referencing context that you will be provided separately. You MUST return a better prompt between <BetterInstruction> and </BetterInstruction> tags. Original Instruction: === {TASK} ===

(107) Here is another example of an LLM prompt used to generate the universal_writer function to perform a writing task.

(108) TABLE-US-00008 You are a direct and straight-forward, and direct AI assistant. You do not ask questions, or follow ups. You just return your best attempt at the task. Assume it's for something general, unless given more information. Provide a direct answer without additional commentary. Put your response between <UniversalWriter> and </UniversalWriter> tags. If you cannot accomplish the task, respond in the <Concerns> and </Concerns> tags. If you are sorry that you cannot do it, respond in the <Concerns> and </Concerns> tags. If the information you need does not exist, respond in the <Concerns> and </Concerns> tags. Always provide your best response. Document === {CONTEXT} === Task === {TASK_IMPROVED_VERSION} === You MUST respond with <UniversalWriter> and </UniversalWriter> tags.

(109) The data_extractor function is used to extract data from any given source. For example, extract inventor names from a patent (given the patent number, which is the context), extract the claims from a patent, extract the products on sale from a website, extract today's appointment from a calendar, etc.

(110) As with universal_writer, the inputs are the task and the context; the output is the extracted information. The format of the output will depend on the task requested (e.g., “generate a markdown table with stock prices” will produce a markdown table). Also, the data_extractor function may generate its own LLM prompt.

(111) Here is an example of an LLM prompt used to generate the data_extractor to use an LLM to copy segments of text that are relevant to the task.

(112) TABLE-US-00009 Copy any information related to: {DATA_KEYS} Document === {CONTEXT} === From the document above, I want snippets that will help me with: {EXTRACTION_TASK} === Please COPY ALL information that is relevant. You MUST INCLUDE supporting content that shows the context.

(113) A second LLM prompt example for universal_writer is to extract data from identified segments and put the output into a structured format. This second example is more complex than the first one, so a cheaper LLM may be used in the first example, while a more expensive (in terms of computing resources required) for the second example. Here is the LLM prompt for the second example:

(114) TABLE-US-00010 You are a data extraction expert. You MUST only use information from provided sources. Document === {CONTEXT} === {EXTRACTION_TASK} === Extract data from the source document, calling save_data with the results in ResultUnion. Required fields:

{REQUIRED_FIELDS} Optional fields: {OPTIONAL_FIELDS} === Please call save_data with the data you extracted. Try your BEST to extract the data, skip optional fields as needed. If the data is not in the source document, please call save_data with failure_reason.

===== Note:

Prompt 2 is called with the latest “function_call” schema that OpenAI provides: class Failure(pydantic.BaseModel): failure_reason: str class DesiredData(pydantic.BaseModel): {DATA_SCHEMA} openai.ChatCompletion.create(... function_schema={ "name": "save_data", "description": "Save extracted data", "parameters": { "title": "ResultUnion", "type": "object", "properties": { "result_union": { "title": "Result Union", "oneOf": [DesiredData.schema(), Failure.schema()], } }, "required": ["result_union"], }, }, ...)

(115) As the code is running, the code is using universal_writer **802** and data_extractor **804** to write and extract data independently. It is figuring out how to use itself inside a program, akin to an inception-like scenario. What is interesting to note is that these functions provide the code with a set of tools for the code to use.

(116) The LLM prompt creation has several unique features, such as prompting using multiple-inputs and multiple-outputs. In some example embodiments, the LLM prompts use XML-based tags to indicate the different types of inputs and outputs, such as:

(117) TABLE-US-00011 Inputs: - <task>please summarize this</task> - <content>this is a long document</content> Outputs: - <summary>this is a summary</summary> - <concerns>these are any concerns that came up while processing the summary</concerns> - <success>true</success>

(118) Furthermore, tags can be randomized to prevent prompt injection attacks that might guess at what tags are used, or if an input/output would happen to also contain the corresponding tag.

(119) FIG. 9 illustrates the execution of actions by the LP executer **902** that is part of the LCIEE, according to some example embodiments. LLMs can generate code **916**, but there is still a need for a programmer or developer to execute the code and operate the environment (e.g., open up a terminal window, run pip, run npm, etc.). The LP executer **902** provides an execution environment for LLM-generated code, where the LP executer **902** closely controls execution for safety and reliability, and automatically self-heals for erroneous code generated by the LLM.

(120) In some example embodiments, there is a custom code interpreter (e.g., python interpreter) for the LLM code to run in. The execution environment is customized to safely make calls to external APIs, have fine-grained control over the code execution state, and analyze the code statically, while fully leveraging the outputs and errors.

(121) The benefits of this approach include being able to statically analyze how input/output sources are connected. Further, by controlling the code interpretation, it is possible to dynamically prevent infinite loops, prevent leakage of API secrets, allow debug/resume/breakpoints, automatically parallelize execution to allow faster iteration, let users do bigger tasks (e.g., handle large batches of data, etc.), and automatically report progress to users to improve the user interface and allow faster iteration.

(122) This approach enables LCIEE to leverage best practices from software engineering to be used in an LLM-created code environment, which includes the following abilities: Support triggers and events as first-class citizens; cache code output and reuse the code in future applications; enable code to be rewritten part-way through execution; use the LLM to interpret user feedback in context of an existing program, update the code to fix problems, and update ongoing execution/results; and support the idea of “exec_natural,” where the LLM can decide that halfway through a program, it does not know what code should execute yet, so it defers execution by telling itself to write the code later based on the context at a future time.

(123) This approach provides an elegant solution to challenges in the planning, memory, composability space that is common in AI research, and the LP executer **902** enables these features

through software engineering concepts, e.g., memory is simply local variables, composability is functions, planning is the script/code, etc.

(124) Further, LCIEE provides effective code generation using a high-level planner. By separating the planning and the code generation, the result is more reliable and better code. As discussed above, the high-level plan is created first, which may include high level references to function names and their purposes, and then the code is generated from the high-level plan.

(125) The LP executor **902** is responsible for interpreting and executing the LLM-generated code and provides fine-grained control over the execution state, enabling safety measures such as preventing infinite loops, leakage of API secrets, and allowing debugging features like breakpoints and resume points.

(126) The LP executor **902** obtains the code **916** and determines an Abstract Syntax Tree (AST **912**) for every line and part of the code **916**. The AST **912** is a tree representation of the abstract syntactic structure of source code written in a programming language. Each node of the tree denotes a construct occurring in the source code, and the syntax is “abstract” in not representing every detail appearing in the real syntax, but rather just the structural or content-related details. ASTs are used in the analysis of source code and are used in the process of program compilation, where they are used to build the structure of the program, helping to understand the syntax of the new language and a step in the translation process to convert high-level language instructions into machine code.

(127) For example, an if statement would have a left branch and a right branch, and the AST **912** will have a left and right branch in the tree representation. The LP executor **902** then proceeds to walk through the code **916**, executing it in a detailed manner at every token level input.

(128) In the illustrated example, at some point, the LP executor **902** reaches an action email get **910** that access an email server to retrieve email for the user. To get the email, the credentials **908** of the user are needed. Other possible actions may include a search action **904** to perform a search, a send-email action **906** to send email, a server-request action **918**, etc.

(129) The user can choose which actions they want to enable for the LP executor **902** to access. For instance, they may choose to enable email get and search, but not sending emails because it may be too risky.

(130) The LP executor **902**, for the actions enabled by the user, is informed during the code generation process about the actions it has access to, but the LP executor **902** is not informed about the actions it does not have access to. Consequently, the generated code **916** is not even aware of the actions it does not have access to.

(131) While the LP executor **902** is running, if an attempt is made to execute an action that is not enabled, the call for that action is simply extracted from the code and not executed.

(132) The credentials **908**, (e.g., passwords, API keys, and other similar elements) are not involved in the code generation, and the credentials **908** are handled in another part of the system separate from the code generation.

(133) Users may wish to integrate the actions they use into the LCIEE. One method would be to write it into the existing code base, but this is not ideal. Instead, a new action is divided into two parts: the API call and the processing of the results from the API. In the illustrated example, the server-request action **918** is divided into the actual API call and the processing **920**.

(134) The API call could involve calling another website or distributed entity with certain credentials. This can be configured to be very generic, as calling a website is a common action. This part is made available so that anyone can enable it. However, the challenge arises after obtaining the results from that API call. Often, these results need to be manipulated or reformatted in a way that makes sense or that is easier to process. Typically, after calling the API, there is some code that transforms the outputs into a more usable form, and it might also be necessary to capture errors. Therefore, the processing **920** includes code that transforms the output. The API call is generic and well defined and understood, but the processing **920** tends to be very specific

depending on the type of request.

(135) The LP executer **902** allows users to add code to the environment, without restrictions, and the code can then be executed within the LP executer **902** in a safe environment. Further, the LP executer **902** may also generate this code and be saved for later reuse.

(136) In some example embodiments, a user may provide the documentation for accessing a resource via an API, and then requests LCIEE to create the code to access this resource based on the documentation. All the user needs to do is to add credentials **908**, if needed, separately.

(137) FIG. **10** is a sample architecture for the LCIEE **1002**, according to some example embodiments. The LCIEE **1002** combines several key concepts from the fields of artificial intelligence, software development, and information security, including, but not limited to, static code analysis, dynamic execution control, auto-healing, parallel execution, and user feedback interpretation. These features are bundled into a cohesive product that aims to enhance LLM's usability, versatility, and safety in coding applications, thus bringing the principles of software engineering into the realm of AI and LLMs.

(138) In some example embodiments, the LCIEE **1002** includes the following components: the LP executer **902**, a plan generator **1012**, a code generator **1016**, an LP analyst **1020**, a prompt generator **1036**, a progress reporter **1038**, a database **1024**, a user UI **1010**, a management module **1030**, a management UI **1028**, and a task marketplace **1026**. In other embodiments, the LCIEE **1002** may include additional components, fewer components, combine components, or break components into multiple elements.

(139) The LP executer **902** is responsible for controlling the overall execution flow, allowing dynamic interventions like code rewrites part-way through execution and interpreting user feedback to adjust execution and results. In some example embodiments, the LP executer **902** includes an execution parallelizer component that works to parallelize code execution, enabling the LCIEE to handle larger tasks and improve execution speed.

(140) The plan generator **1012** produces the detailed high-level plan from the user prompt, and the high-level plan is then used by the code generator **1016** to produce the code. The code generator **1016** passes the code **1018** to the LP executer **902**.

(141) Here is an example of code generated by the code generator **1016** for a request to analyze big tech companies and store the information in a Google Sheet.

(142) TABLE-US-00012 # Analyze big tech companies and store information in a Google Sheet.

```
def analyze_big_tech_companies( ):    # Define a list of big tech companies to analyze.
companies = ["Apple", "Microsoft", "Amazon", "Google", "Facebook"]    # Initialize a list to
store the company information and news summaries.    company_data = [ ]    # Iterate through
each company in the list.    for company in companies:        # Obtain the company's stock ticker
symbol.        ticker_result = finance_get_ticker(company)        if not ticker_result.success:
continue ...
```

(143) The LP executer **902** will check if the code **1018** includes comments, and these comments may be used as descriptors of the functions or actions being executed. Because the comments are usually high-level descriptions that regular users can understand, the LP executer **902** brings the comments to the UI **1010** via the progress reporter **1038** (see for example the UI **602** in FIG. **6**). The UI **1010** is presented on a user device **1004** associated with user **1006**. For example, the prompt **1034** is provided by the user **1006** on the UI **1010**.

(144) Further, if an error **1032** occurs during execution, the LP executer **902** sends the error back to the code generator **1016**, which examines the error **1032** and attempts to generate improved code **1018** for the LP executer **902** to try again for that particular function or action. The process may be repeated several times (e.g., up to three times or more) before the LP executer **902** stops trying and reports the error to the UI **1010**.

(145) The user and the LCIEE may also create triggers **1022**, which may be scheduled (e.g., give me a summary of the news every morning at 7 AM) or that are generated based on events (e.g.,

wake up the system analyst when there is a computer crash). Each trigger will cause the execution of a task, a function, or an action by the LP executer **902**.

(146) The LP analyst **1020** scans the LLM-generated code to understand how input/output sources are connected, aiding in the optimization of the code and its execution. The information is used to inform the user about the actions the system might take, such as accessing their emails. Thus, the LP analyst **1020** functions between the code generator **1016** and the LP executer **902**. For example, the LP analyst **1020** generated the ambiguity field **206** and the tools field **208** shown in FIG. 2.

(147) The prompt generator **1036** generates the LLM prompts used by the code generator **1016** to feed as inputs for the LLM. There could be multiple types of prompts being generated, and in some example embodiments, a plurality of prompt generators may be available, each with a distinctive goal to generate LLM prompts for a specific purpose.

(148) The progress reporter **1038** tracks and reports progress and partial results to the user to improve the user experience and allow faster iterations. The progress reporter **1038** prepares information provided by the LP executer **902** for the user, such as progress reports, results of the execution of the plan, and commands being executed.

(149) The database **1024** may include a single database or a distributed database system to store different parts of the stored data. The database **1024** stores code, plans, results, API calls executed, progress reports, errors, actions, credentials, and other LCIEE-related data. The database **1024** may be accessed by the different components of the LCIEE **1002**.

(150) The user UI **1010** is used for interfacing with the user, such as the UIs in FIGS. 1-2, and others. The UI **1010** may also describe what tools are available to the user, and possibly which tools are available if the user obtained authorization to access them.

(151) The management module **1030** is used for configuring management options, such as adding users, providing access to users, enabling functions for the LP executer **902**, configuring which tasks can be used by which users, etc. The management module **1030** interacts with an administrator device **1008**, associate with administrator **1014**, via the management UI **1028**.

(152) Once a plan is created and properly executed, the LCIEE provides an option to save the plan to make it reusable for the future. As discussed above, the saved plan is referred to as a task, and these tasks are saved in the task marketplace **1026**, so the user, and other users, may invoke them in the future. This operation is referred to as the “Export and Save” function. More details about the use of the task marketplace **1026** are provided below with reference to FIG. 14.

(153) FIG. 11 is a flowchart of a method **1100** for processing a request by the LCIEE, according to some example embodiments. While the various operations in this flowchart are presented and described sequentially, one of ordinary skill will appreciate that some or all of the operations may be executed in a different order, be combined or omitted, or be executed in parallel.

(154) Operation **1102** is for providing a UI for entering the user prompt, showing progress, and other plan-related options. Additionally, a trigger may request that a plan or task is executed at operation **1116**.

(155) From operation **1102**, the method **1100** flows to operation **1104** to create, by the plan generator, a plan with operations, and present the plan in the UI to the user, which allows the user to provide additional information if needed to complete the plan and permissions to access certain tools if needed.

(156) From operation **1104**, the method **1100** flows to operation **1106** to create the code by the code generator. The code generator may invoke the LLM one or more times to generate code.

(157) From operation **1106**, the method **1100** flows to operation **1108** to check the code by the LP analyst.

(158) From operation **1108**, the method **1100** flows to operation **1110** execute the code by the LP executer. If the LP executer runs into an error during the execution of the code, the code generator will retry to generate new code to try execution again, up to a number of times until the LP executer quits.

(159) The LP executor **902** may perform actions (operation **1118**) to access external servers.

(160) From operation **1110**, the method **1100** flows to operation **1112** to present commands, progress, and results in the UI.

(161) Optionally, the completed plan may be saved as a task in the task marketplace at operation **1114**.

(162) FIG. **12** illustrates the generation of a UI utilizing LCIEE, according to some example embodiments. It has been observed that with a recent version of ChatGPT, if a description of the user interface is provided, ChatGPT can generate a user interface in code.

(163) Given a scenario where a textual description of the desired user interface is provided (e.g., provide a UI with this title, this message, and two input fields named user_name and password), the LLM can generate the corresponding code, e.g., a website, complete with HTML, JavaScript, and CSS. However, there are certain issues involved, such as how is the code integrated into the system and in what website is the UI implemented.

(164) LCIEE provides a way to enter a request to implement an UI, via the prompt **1202**, to allow the user to provide a high-level description of the user interface, and then generate the code **1206** using the LLM **1204**. Further, LCIEE creates the connections that link the UI into the APIs available for access. This results in aesthetically pleasing and functional interfaces.

(165) The UI needs to be executed, and this execution may occur in the LCIEE. The code can be generated, stored, and served, by the LCIEE. That is, LCIEE takes the user interface request and generates the entire backend to supports the UI. This is referred to as the backend LCIEE. Further, LCIEE can also provide data storage for the user.

(166) The created UI may interface directly into a user's own database that may exist outside the LCIEE. The connection may be established through a plan or through a task that has been saved in the task marketplace, where the task is linked back to an external API provided by the user. The task exposes the API, and there is an action in the user interface that calls for the task to be executed.

(167) For instance, if a UI is designed to search the US Patent Office database, the user can request a user interface with various fields. When the search is conducted for a specific client, the UI would cause LCIEE to access the database and execute the search.

(168) FIG. **13** is a flowchart of a method **1300** for utilizing the task marketplace, according to some example embodiments. Some plans are repeatable, that is, after being executed one time properly, the plan could be performed again in the future. For instance, a search that locates products with features similar to a given patent (e.g., to search for potential infringers).

(169) It would be nice not having to start a plan from scratch and reuse the code that has already been successfully executed. Further, some aspects of the plan may vary from plan to plan (e.g., the patent number), while everything is the same. In this case, LCIEE provides a feature to name variables to be embedded in the plan when it is saved as a task. What varies in the code of the task is the argument, which is the patent number that is passed to execute the search.

(170) LCIEE enables plans to be reusable when saved as tasks, and these tasks can have inputs and outputs. Users can ensure plans work, save the executed plans as tasks, and then share these tasks with colleagues. The colleagues can then modify the inputs to suit their needs.

(171) Further, when a plan uses one or more of the saved tasks, the new plan may be itself converted into a new task. This way, the user can continue to increase functionality based on the building blocks, i.e., the tasks, previously created. For example, a task is created for creating a mobile app, and another task was created to access health records. The tasks can then easily be combined to create a mobile app that accesses health records.

(172) Additionally, LCIEE provides some system-created tasks that can be used by other users, such as the previously described universal_writer and data_extractor.

(173) The significant advantage of LCIEE is its potential to automate repetitive tasks. Many individuals have jobs that involve repetitive tasks. The aim is to examine these processes and

identify areas where efficiency can be improved, and the goal is not necessarily to replicate the exact process, but to automate tasks that consume a significant amount of time. By spending a few hours setting up this automation, individuals can reclaim those hours for other tasks.

(174) For instance, a user could create a system where dropping a document into a specific OneDrive folder triggers an email to be sent after 20 minutes. The question then becomes whether one can optimize their life significantly using these automated processes. Further, as the system evolves and grows, it becomes more efficient and supports increasing functionality.

(175) Returning now to the method **1300**, operation **1302** is for providing an option in the UI to save an executed plan, as a task, for future use.

(176) From operation **1302**, the method **1300** flows to operation **1304** where the task information is saved in the database.

(177) From operation **1304**, the method **1300** flows to operation **1306**, where an option is provided in the UI to allow the user to access the task marketplace and use tasks of the user and tasks enabled by other users.

(178) From operation **1306**, the method **1300** flows to operation **1308** where a selection of a task from the task marketplace is detected.

(179) From operation **1308**, the method **1300** flows to operation **1310** to perform the selected task for the requesting user.

(180) FIG. **14** is a sample UI **1402** for accessing the task marketplace, according to some example embodiments. One goal is for individuals to publish their tasks for others to use, creating a marketplace of saved tasks. While the creator of the task may be the primary user, other users can also benefit from these tasks.

(181) In the task marketplace, users who understand how these tasks operate can utilize them. The LCIEE allows for the execution of tasks without the need to view the code or the plan behind the task.

(182) In the illustrated example, the UI **1402** shows a list of available tasks in a table **1404**, and the user can scroll down the page to see additional tasks. For each available task, the corresponding row in the table **1404** includes the task name, a task description, input (if any) for the task, output expected after executing the task, and the creator of the task. Additional information may also be included, such as when the task was created, last time the task was executed, number of times the task has been executed, dependencies (e.g., access to email service), etc.

(183) Some of the tasks are provided by the system (e.g., `universal_writer` and `data_extractor`), and other tasks have been created by users. When a user selects a task, a new UI will be presented with information about the task and options to enable the user to enter required inputs, credentials, etc. Further, an option is provided to request execution of the task, and when selected, the task will be executed by the LCIEE.

(184) FIG. **15** illustrates the option to generate additional code during the execution of the code and then executing the additional code, according to some example embodiments. Another feature of LCIEE is ability to generate code “on the fly” (e.g., while the code is being executed) and run the generated code without having to interrupt the running code.

(185) In the illustrated example, code **1502** is being executed, and the code **1502** includes a list of operations (e.g., function 1, function 1), with their corresponding comments.

(186) There is a special function named `generate_lp_code` that allows LCIEE to generate code and another special function named `run_lp_code` that can execute the code generated by `generate_lp_code`. That is, the output of `generate_lp_code` is a block of code, and the input to `run_lp_code` is the code.

(187) For example, an operation could be to generate code to “find patents associated with heart medical inserts.” The code generator would be used to generate this code, and then the code generated would be executed by the LP executer **902**.

(188) Thus, LCIEE can, during execution, generate code and run that generated code recursively

within itself.

(189) FIG. **16** is another example of the generation of the plan. Some prompt may ask for requests that are very complex, so LCIEE may perform several cycles for generating operations, where an operation generated in one cycle may be expanded into a list of sub-operations in the next cycle, until the code is generated for some of the operations and sub-operations.

(190) As used herein, an operation that is submitted for the creation of code is referred to as an atomic operation, while operations that are used as a new prompt to generate a sequence of suboperations are referred to as decomposable operations.

(191) In the illustrated example, the user has submitted an ambitious request, “Think of ideas for a world-changing impactful app, then pick one. You will implement it today. You can make all decisions, do a great job!” LCIEE generated the following plan: 1. Generate a list of 10 well-changing impactful app ideas. 2. Gather information about existing solutions and competitors for each app idea. 3. Analyze the search results and evaluate the potential impact, feasibility, and uniqueness of each app idea. 4. Rank the app ideas based on the analysis from step 3. 5. Select the top-ranked app idea as the final choice. 6. Create a document outlining the chosen app idea, its potential impact, and the reasons for selecting it. 7. Create a detailed implementation plan for the chosen app idea. 8. Send a chat message to the CEO with a summary of the chosen app idea, its potential impact, and a link to the implementation plan document. 9. Begin executing the implementation plan as outlined in the document.

(192) It can be seen that many, if not all, of these operations are complex so they can be decomposed into simpler operations, e.g., operation 7 that recites “Create a detailed implementation plan for the chosen app idea.”

(193) FIG. **17** shows a document generated by the plan, according to some example embodiments. With the plan described in FIG. **16**, LCIEE generated three documents: a first document with app ideas, a second document with the chosen idea (shown in part in FIG. **17**), and a third document with the implementation plan (shown in FIG. **18**). The documents may be saved in any format that supports text, such as Microsoft Word, Google doc, plain text, RTF, etc.

(194) In addition to the three created documents, the initial prompt presented in FIG. **16**, generated the following results in the UI describing the chosen app idea: Summary of chosen app idea: The app is a platform that connects volunteers with local organizations in need of help. Its main features include a search function for volunteer opportunities, a messaging system for communication between volunteers and organizations, and a tracking system for volunteer hours. The target audience is individuals who are interested in volunteering and local organizations that need volunteers. The potential impact of the app is significant, as it could increase volunteerism and help organizations fill their volunteer needs more efficiently. However, the app may face challenges in ensuring the safety and reliability of volunteers and organization.

(195) In this example, LCIEE came up with the following list of impactful app ideas described in the first document: 1. An app that connects volunteers with local organizations in need of help 2. A mental health app that provides resources and support for those struggling with mental illness 3. An app that helps people reduce their carbon footprint by tracking their daily habits and suggesting eco-friendly alternatives 4. A language learning app that connects native speakers with learners for conversation practice 5. An app that helps people find and book affordable housing in their area 6. A food waste reduction app that connects users with local food banks and charities 7. An app that helps people track and reduce their screen time for improved mental health . . .

(196) After generating these ideas, each idea is treated as a new prompt, and LCIEE explores the idea. For example, LCIEE performs research on the feasibility and plans for implementing the idea, as well as the potential impact.

(197) Eventually, LCIEE selects one of the ideas, as defined in operation 5 of the first-level plan, and creates the second document as defined in operation 6 to describe the chosen idea. Part of the second document is illustrated in FIG. **17**. In this example, the second document describing the

chosen app includes the following: Chosen App Idea: 1. An app that connects volunteers with local organizations in need of help Potential Impact: Based on the provided information, there are already several establish platforms and apps that connect volunteers with organizations, such as VolunteerMatch POINT, and Golden Volunteer Opportunities. However, there is still potential for a new app that focuses on connecting volunteers with virtual opportunities, as seen with Volunteer From Your Couch. The target audience would be individuals looking to donate their time and skills to various causes. The app should have features that allow users to easily search for and apply to volunteer opportunities, track their volunteer hours, and communicate with the organizations they are working with. The revenue model could be based on a subscription fee for organizations to post volunteer opportunities or a percentage of donations made through the app. The main challenge would be standing out among the existing competition and gaining traction in the market. However, with a unique and user friendly interface, effective marketing strategies, and partnerships with reputable organizations, the app could have potential for growth and success. Purpose: To connect volunteers with local organizations in need of help. Target audience: Individuals looking to volunteer in their community. Features and functionalities: User registration and login Volunteer opportunities search and filtering Volunteer opportunity details and sign-up Organization registration and login Organization profile creation and management Volunteer management and communication In-app messaging system Rating and review system Push notifications Timeline: Planning and research: 1 month Design and prototyping: 2 months Development: 6 months Testing and quality assurance: 1 month Development team roles and responsibilities: Project manager: oversees the entire project and ensures deadlines are met . . . Link to implementation plan: [implementation_plan.docx](#)

(198) As seen in the last line, a separate document has been created for the implementation plan and a link is provided.

(199) FIG. **18** provides details on the implementation plan, according to some example embodiments. The implementation plan includes the following: The plan is to: 1. Generate search queries related to user registration and login best practices. 2. Perform a general search using the generated queries. 3. Extract URLs from the search results. 4. Get the content of each website and summarize the best practices for user registration and login. 5. Save the summaries in a document. 6. Rank the best practices based on their relevance and effectiveness. 7. Select the top-ranked idea. 8. Create an outline for implementing the final choice. 9. Save the outline in a document. 10. Generate an implementation plan based on the outline. 11. Save the implementation plan in a document. 12. Create a summary of the implementation plan. 13. Break down the implementation plan into a list of tasks. 14. Send the tasks to the user. 15. Return the summary. . . .

(200) Further, in this example, LCIEE generates a plan to execute the implementation plan, as follows: The plan is to: 1. Read an implementation plan document. 2. Extract a list of tasks from the implementation plan document. 3. Split the list of tasks into individual tasks. 4. Execute each task using natural language processing. 5. Send a chat message to the user indicating that each task has been completed. 6. Send a final chat message to the user indicating that the implementation plan has been successfully executed. Reading document at [implementation_plan.docx](#) Generating content for task: extract a list of tasks . . .

(201) Then LCIEE continues with the execution by performing the different operations within the plan. For example, one of the operations in the plan is related to user registration and login best practices. LCIEE generated the following document regarding login best practices: Here are some best practices for user registration and login: 1. Allow sign-up with social media accounts for one-click registration. 2. Don't ask for password confirmation. 3. Use password strength indicators to encourage users to create strong passwords. 4. Implement multi-factor authentication for added security. 5. Use HTTPS to encrypt user data during transmission. 6. Implement rate limiting to prevent brute force attacks. 7. Use password managers to securely store user passwords. 8. Provide clear and concise error messages to help users troubleshoot issues. 9. Use user-friendly interfaces

that are easy to navigate. 10. Consider accessibility considerations for user with disabilities. 11. Implement a secure password reset process that requires additional verification. 12. Manage user data securely and transparently, following data protection regulations.

(202) LCIEE proceeds to perform all these operations until the complete plan is determined. As described, LCIEE generates plans that grow in specificity, going from more general to more specific. Some of the plans are complex, so LCIEE recursively uses some operations like a prompt to break down a complex operation into simpler operations, until all the operations may be performed executing code.

(203) The benefit of LCIEE recursively improving a plan is that LCIEE retains the environment that the code is running on, including any internal variables and other functions used in the process.

(204) FIG. 19 is a flowchart of a method 1900 that uses recursiveness to expand operations into a plurality of sub-operations, according to some example embodiments. Recursion is a method of solving a problem by breaking the problem into smaller subproblems of the same type. A recursive function calls itself until one or more times until it reaches a state that may be solved without recursion, i.e., without having to call the recursive function again.

(205) Operation 1902 is for receiving a prompt from the user. From operation 1902, the method 1900 flows to operation 1904 where a plan is created, and the plan includes operations. The operations could be atomic operations or decomposable operations that can be used to generate other sub-operations.

(206) From operation 1904, the method 1900 flows to operation 1906 to execute the operation. Within operation 1906, operation 1910 is used to perform a check to determine if the operation being executed should be expanded. If the operation is an atomic operation and does not have to be expanded, the method flows to operation 1914 to execute the atomic operation, which is considered the end (operation 1916) for executing the operation 1906.

(207) An atomic operation is an operation that can be executed by creating code using the LLM. Non-atomic operations are those operations that are considered too complex to be able to generate code, using the LLM, to execute the operation. In some example embodiments, an operation is deemed to be non-atomic after failing to successfully execute code, created by the LLM for the operation, a predetermined number of times. In other embodiments, the operation is deemed to be non-atomic when the size of the code, created by the LLM for the operation, exceeds a predefined size (e.g., 1 Mb, but other thresholds are possible). In yet other embodiments, the operation is deemed to be non-atomic when the LP Analyst determines that the code, created by the LLM for the operation, will not execute successfully.

(208) At operation 1914, to execute the atomic operation, code is generated for the atomic operation and then the generated code is executed. One advantage of generating the code when the atomic operation is to be executed is that the system can delay the generation of code until the code is going to be executed. By delaying code generation, the code can work with the context at that time, the context comprising functions, parameter values, and data used and generated by previously executed code. The system is able to delay generating the code so when the code is generated, the complete context at that time can be used to generate the code and execute the code.

(209) This is important for complex plans, as the system goes through expansions of complex operations, so execution of some operations may be delayed until the plan is more refined after several expansions of the initial plan with the initial sequence of operations.

(210) However, if the operation is a decomposable operation that needs to be expanded, the method flows to operation 1912 to create a new sequence of operations to perform this operation, that is, create sub-operations. All the created sub-operations are then executed recursively within operation 1906, which in turn could be decomposable or atomic.

(211) After all the recursions are performed and all the operations executed, the method 1900 flows to operation 1908 to return the results to the user and present the results in the UI.

(212) FIG. 19 shows that the operations of a plan can be recursively used to generate sub-

operations (also called subplans) and then continue with the sub-operations. This way, very complex tasks can be decomposed in smaller, simpler tasks that are easily coded for execution.

(213) FIG. **20** is a flowchart of a method for implementing a tool that generates and executes the code in a controlled environment to satisfy user requests entered as text prompts, according to some example embodiments.

(214) Operation **2002** is for receiving a user prompt with a user request received in a UI.

(215) From operation **2002**, the method **2000** flows to operation **2004** to generate generating a first LLM prompt to create a plan to perform the user request.

(216) From operation **2004**, the method **2000** flows to operation **2006** for receiving the plan from an LLM that received the first LLM prompt, the plan comprising text describing a sequence of operations.

(217) From operation **2006**, the method **2000** flows to operation **2008** to generate a second LLM prompt to create code, the second LLM prompt specifying which code instructions are permitted in the created code.

(218) From operation **2008**, the method **2000** flows to operation **2010** for receiving the code from the LLM that received the second LLM prompt, the code comprising instructions executable by a computer processor.

(219) From operation **2010**, the method **2000** flows to operation **2012** for executing the code in a controlled environment, the executing the code comprising detecting an error when executing a function of the code, generating new code for the function that caused the error, and executing the new code for the function that caused the error.

(220) From operation **2012**, the method **2000** flows to operation **2014** to cause presentation in the UI of results generated by executing the code.

(221) In one example, the method **2000** further comprises detecting that credentials to access an external tool are required to satisfy the user request, and presenting in the UI a request to obtain the credentials.

(222) In one example, the method **2000** further comprises detecting that a parameter value is needed to satisfy the user request, and presenting in the UI a request to obtain the parameter value.

(223) In one example, the parameter value is an email address, wherein the UI presents a request for the email address.

(224) In one example, the method **2000** further comprises providing feedback on the UI about progress while executing the code, the feedback comprising information based on comments associated with function calls in the code.

(225) In one example, the first LLM prompt comprises: information on data classes, functions, and libraries that can be referenced, tools available to be used in the plan, instructions for the LLM to generate the plan, and the user request.

(226) In one example, the second LLM prompt comprises: information on data classes that can be referenced, libraries available, functions that may be used in the code, instructions for the LLM to generate the plan, and the plan.

(227) In one example, the first prompt provides instructions to use a text generation function with inputs of a task to be performed and a context and generates an output with information requested in the inputs.

(228) In one example, credentials required to access resources excluded from the code.

(229) In one example, executing the code further includes executing a function to obtain information from another computer using an Application Programming Interface (API), and transforming the information received in response to executing the function based on requirements to execute other functions within the code.

(230) Another general aspect is for a system that includes a memory comprising instructions and one or more computer processors. The instructions, when executed by the one or more computer processors, cause the one or more computer processors to perform operations comprising: receiving

a user prompt with a user request received in a user interface (UI); generating a first Large Language Model (LLM) prompt to create a plan to perform the user request; receiving the plan from an LLM that received the first LLM prompt, the plan comprising text describing a sequence of operations; generating a second LLM prompt to create code, the second LLM prompt specifying which code instructions are not allowed to create the code; receiving the code from the LLM that received the second LLM prompt, the code comprising instructions executable by a computer processor; executing the code in a controlled environment, the executing the code comprising: detecting an error when executing a function of the code; generating new code for the function that caused the error; and executing the new code for the function that caused the error; and causing presentation in the UI of results generated by executing the code.

(231) In yet another general aspect, a non-transitory machine-readable storage medium (e.g., a non-transitory storage medium) includes instructions that, when executed by a machine, cause the machine to perform operations comprising: receiving a user prompt with a user request received in a user interface (UI); generating a first Large Language Model (LLM) prompt to create a plan to perform the user request; receiving the plan from an LLM that received the first LLM prompt, the plan comprising text describing a sequence of operations; generating a second LLM prompt to create code, the second LLM prompt specifying which code instructions are not allowed to create the code; receiving the code from the LLM that received the second LLM prompt, the code comprising instructions executable by a computer processor; executing the code in a controlled environment, the executing the code comprising: detecting an error when executing a function of the code; generating new code for the function that caused the error; and executing the new code for the function that caused the error; and causing presentation in the UI of results generated by executing the code.

(232) FIG. 21 illustrates the process of creating a task for the marketplace, according to some example embodiments. Users may want to share their tasks and make them available in the task marketplace for other users to execute. However, the sharing user wants to ensure there are no security risks by sharing tasks. For example, if the user creates a task that includes sending an email, the task will access the user's credentials to send the email. But if the user shares the task, the user wants to make sure that the code will not access the user's credentials to send email, that is, that the access to the email account of the sharing user is completely protected. As used herein, a credential is information required to access a resource.

(233) On the other hand, a user who wants to use a task created by another user wants to make sure that this task will not be a security threat to the user. For example, if the task accesses a file in a server, the user wants to ensure that the access to the file will not be shared with other users. That is, the credentials of the user to access files will never be shared with other users.

(234) Other solutions share code by completely locking the code during execution and limiting the operations that can be performed, such as by disallowing access to files from the code. However, this type of solution greatly limits the possibility of sharing tasks that perform various operations, including accessing external resources requiring credentials.

(235) In the illustrated example, the user executes a plan **2102**, and part of the plan includes accessing an input file **2104**, which requires credentials **2106** to access this file, e.g., credentials to access a Google Drive document.

(236) The plan is processed **2108**, which includes generating an output file **2110** using file credentials **2112**, which may be the same or different from credentials **2106**. For example, the results will be stored in the output file **2110** in Google Drive.

(237) A token is a short string of characters used to authenticate a user or application and grant access to a protected resource. In the context of Google Drive, a token can grant an application access to the user's Drive account.

(238) A token is an item that allows access to a resource owned by another user without knowing the name and password of the user sharing access. For example, Google Drive uses two main types

of tokens: access tokens and refresh tokens. Access tokens are short-lived tokens used to grant an application access to the user's Drive account for a limited period, e.g., one hour. Refresh tokens are long-lived tokens that can be used to obtain new access tokens when the current access token has expired. Refresh tokens are typically obtained when the user first grants access to their Drive account.

(239) To use a token to access Google Drive, the application must first make an HTTP request to the Google Drive Application Programming Interface (API). The Google Drive API will verify the token and grant the application access to the user's Drive account.

(240) Some examples of how tokens can be used to access Google Drive include: a web application can use tokens to allow users to browse and manage their Google Drive files; a desktop application can use tokens to allow users to sync their local files with their Google Drive account; a mobile application can use tokens to allow users to access their Google Drive files on the go, and a server-side application can use tokens to access and process Google Drive files for automated tasks.

(241) After a successful execution of the plan, the user saves the plan as a task **2114** in the task marketplace. The task **2114** includes the code **2116** that was executed and the credentials **2118** required to access external resources. In this example, the credentials **2118** would include the credentials **2106** and the file credentials **2112**.

(242) FIG. 22 illustrates the creation of a task that combines multiple tasks, according to some example embodiments. Plans may include executing one or more tasks. For example, a user has executed a plan with task **2114** (including code 1 and credentials 1) and another task **2202** (including code 2 and credentials 2).

(243) The user then saves this plan as a new task 3 **2204**, which will include the code and credentials from the task **2114** and the task **2202**, that is, code 1, code 2, credentials 1, and credentials 2.

(244) When a different user runs task 3 **2204**, the other user will provide its credentials to access the different external resources. That is, this user will access her files using her credentials and not access the files of the users who created the tasks and used their own credentials.

(245) By guaranteeing that users access their own resources using their own credentials and that other users will not access their resources and their credentials, a level of trust is created for the user who knows that her resources are safe and will stay private.

(246) The code and the credentials are separated and handled separately. Because executing code from a third party may be scary for some users, the LCIEE **1002** separates the code from the credentials so that the credentials can be managed during run time. The LCIEE **1002** accesses the credentials but not the code itself. For example, code that would try to read the credentials and then send them to a malicious user would not be able to do so because the code never gets access to the credentials, as described in more detail below with reference to FIG. 23.

(247) FIG. 23 provides details on the architecture of the LCIEE **1002** for managing access to external resources (e.g., resource **2306**), according to some example embodiments. One objective is to establish a reliable and trustworthy mechanism for handling the submission and access of content by unknown individuals. The system separates the publishing process and associated credentials or tokens to achieve this. This ensures that the person submitting the content does not have visibility or access to the credentials, thereby enhancing the trustworthiness of the published content.

(248) As described above, the LP executor **902** executes the code in a safe environment. To perform critical operations that require credentials (e.g., to access an external resource **2306**), these critical operations are performed separately by a trusted server **2302**.

(249) By employing this architecture, the system establishes a chain of trust, where the code executed within the sandbox environment can securely interact with external services or resources through the trusted server **2302**. This ensures that the publishing process remains secure and trustworthy, as the credentials and tokens associated with the content are never exposed to the

person submitting the content.

(250) When the code executing in the LP executer **902** requests to access the resource **2306** (e.g., an API call to access a Google Drive file), the LP executer **902** sends a request to the trusted server **2302** to perform the operation on its behalf.

(251) The trusted server **2302** keeps a database **2310** of encrypted tokens **2312** to access the different resources by the different users, such that different tokens are used to access the same resource by different users. The trusted server **2302** sends the request to the resource **2306** using the encrypted token **2312** of the user **1006** that requested the plan to be executed, and the response (e.g., data obtained from the resource) is sent back to the LP executer **902**. Therefore, the code executing on the LP executer **902** never has access to the encrypted tokens **2312** used to access the external resource **2306**. The encrypted tokens **2312** are obtained when the user creates the plan to be executed, such as the tools in tools field **208** of Patent disclosure. The user **1006** provides the information to generate the token to access the resource **2306**.

(252) A resource controller **2304** creates sandboxes and manages their operation. The resource controller **2304** is responsible for setting up the execution environment for programs run by the LP executer **902**.

(253) A sandbox is a safe and controlled environment for running programs that enables users to run programs or open files without affecting the application, system, or platform where the sandbox runs. When running code in a sandbox, the sandbox limits the resources that can be accessed by the executing code (e.g., disabling access to memory other than the memory allocated in the sandbox, disabling network access, performing certain operating system operations, etc.).

(254) Direct network access is locked for the LP executer **902**, and the network access has to be performed via a request to the trusted server **2302**. This locking of the network access is a strong security message because the LCIEE **1002** can control which data can leave the LCIEE **1002**, as the network access is controlled by the trusted server **2302**, which runs code managed by the system, that is, the code has been designed and tested by the developers of the system, instead of having code, created by a user, that has not been tested and approved by the developers. This way, the LCIEE **1002** can protect both the credentials and the data processed (e.g., the data that may leave the sandbox).

(255) During network access, security measures may be implemented, such as disallowing access to URLs that are not considered safe, accessing a resource that is not owned by the user executing the task (e.g., saving data to a folder on a cloud service owned by a different user), checking that the data transferred does not include personal identifiable information (PII), etc.

(256) Further, while generating the code to execute a plan based on a user prompt, the LCIEE **1002** may perform checks on the code created to disallow code that performs certain operations or to ensure that the code invokes the trusted server for accessing external resources, e.g., network access. For example, any code that would access the network would be flagged as code that will have to use the trusted server. For example, a function to access a network resource in the code may be replaced with another function that interfaces with the trusted server to perform said network access. In other embodiments, the network calls may be detected during code execution, and the LCIEE **1002** will interface with the trusted server to perform the network calls.

(257) The ownership of the sandboxes is addressed by using unique sandbox IDs for each sandbox. A sandbox database **2308** stores the sandbox IDs created and the information associated with each sandbox ID. The access privileges granted to each sandbox are determined by the resource controller **2304** (e.g., read and write capabilities for Google Docs).

(258) In prior solutions for safe execution of code, users have to trust the application developers that create the code completely. However, using the techniques described herein, the credentials are not known by the executing code, so malicious code cannot take advantage of users. Users only provide tokens to the LCIEE **1002** to access resources, but the user does not have to share passwords to allow access to resources. By using the LCIEE **1002**, users can now trust any code,

independent of the source of the code, because the LCIEE **1002** controls the credentials and does not allow the code to get critical user information. Users can access the task marketplace **1026** and have the confidence to run any of the available tasks shared by other users because the LCIEE **1002** holds the “key” to critical resources.

(259) The LCIEE **1002** eliminates the need for direct credential sharing between developers. Instead, developers provide their credentials to the platform, which assumes the responsibility of managing and safeguarding the credentials on their behalf. Further, users who want to share tasks with other users have the confidence that no harm can be done by sharing their code because the LCIEE **1002** protects their information. Further, the user does not have to load code into the user's device because the code executes in the LCIEE **1002**. By keeping the code away from the user's device, there is a guarantee of safety for the user's device.

(260) An example of code for a task is presented below. In this example, the code includes the instructions for accessing a Google Drive;

(261) TABLE-US-00013 # Define the model for the data to be extracted from the 5498-SA form.
Form5498SA = pydantic_create_model(“Form5498SA”, account_holder_name=(str, ...),
account_number=(str, ...), type_of_account=(str, ...), total_contributions=(
float, ...), fair_market_value=(float, ...)) # Define the function to extract the relevant fields
from the 5498-SA form. def extract_fields_from_5498_SA(pdf_url: str) -> Dict[str, str]: #
Extract the file ID from the Google Drive URL. file_id: str = pdf_url.split('/') [-2] # Retrieve
the PDF file from Google Drive. google_file_info: GoogleFileMetadata =
GoogleFileMetadata(id=file_id, name=‘’, mime_type=‘application/pdf’) try:
web_content: WebContent = get_google_drive_file(google_file_info) except Exception as e:
raise RuntimeError(“Unable to retrieve the PDF file from Google Drive.”) from e #
Extract the relevant fields from the PDF content. try: extracted_data_json: str =
data_extractor(“Extract the account holder's name, account number, type of account, total
contributions made in the year, and the fair market value of the account.”,
web_content.content_markdown, Form5498SA.schema_json()) extracted_data:
Form5498SA = Form5498SA.parse_raw(extracted_data_json) except Exception as e:
raise RuntimeError(“Unable to extract data from the PDF.”) from e # Return the extracted
data as a dictionary. return extracted_data.dict()

(262) In this code, the functions get_google_drive_file and data_extractor are executed by the trusted server **2302**. The function get_google_drive_file gets the content of the file, and the function data_extractor is used to extract relevant fields from the retrieved file.

(263) In some example embodiments, the code for the function get_google_drive_file is as follows:

(264) TABLE-US-00014 async def google_doc_to_markdown_via_html(document_id: str,
max_attempts=3) -> MarkdownDocument: credentials: Credentials =
get_logged_in_user_credentials_for_google() # Initialize the Google Drive API client
drive_service = build(“drive”, “v3”, credentials=credentials) # Export the document to HTML
request = drive_service.files().export(fileId=document_id, mimeType=“text/html”)
html_content = await utils.to_thread(request.execute) # Convert HTML to Markdown return
html_to_markdown(html_content.decode(“utf-8”, errors=“replace”))

(265) The code for the function data_extractor is similar, except that the credentials used are for OpenAI, which, in one embodiment, is the API used for data extraction using the LLM.

(266) FIG. **24** shows the flow of operations to access external resources while executing code, according to some example embodiments. Initially, the user **1006** sends a request **2402** to the resource controller **2304**, asking to run a task associated with the code to be executed. The user **1006** also provides credentials as needed to access external resources.

(267) The resource controller **2304** creates a sandbox with the corresponding sandbox ID and a secret code for executing this task. The resource controller **2304** sends **2404** the sandbox ID and the secret to the trusted server **2302**, and the trusted server **2302** initializes the new sandbox.

Additionally, the resource controller **2304** sends **2406** the sandbox ID and the secret to the LP executer **902**.

(268) Further, the resource controller **2304** sends **2408** the user credentials for this sandbox ID to the trusted server **2302** so the trusted server **2302** can access the required resources when needed. Further, the resource controller **2304** sends a request **2410** to start executing the code to the LP executer **902** in the identified sandbox ID.

(269) The LP executer **902** starts executing **2412** the code. While running the code, updates **2416** may be sent to the resource controller **2304** with information on the progress of the task, and these updates are sent **2432** to the device of the user **1006**. Additionally, the user may cancel the run (not shown), and the resource controller **2304** will initiate the shutdown of the task.

(270) While the code executes, there may be one or more requests to access an external resource **2414**. To access the external resource, the LP executer **902** sends a request **2418** to access the external resource to the trusted server **2302** with information on the sandbox ID and what is required from the external resource **2306**.

(271) When the trusted server **2302** receives the request for accessing the resource **2306**, the trusted server **2302** sends a resource request **2420** to the resource **2306**. The resource sends a response **2422**, with the results of the request (e.g., data, get a file, write to a file, confirmation that an operation was performed), to the trusted server **2302**, and the trusted server **2302** forwards **2424** the response to the code executing in the LP executer **902**.

(272) The above process for accessing the external resource **2306** may be repeated as needed while executing the code. Further, although one resource is illustrated in this example, there may be multiple resources that have to be accessed while the code is executed, and the trusted server **2302** will interface with the multiple resources as needed.

(273) After the code execution, the LP executer **902** sends a termination message **2426** to the resource controller **2304**, and the resource controller **2304** sends a terminate request **2430** to the trusted server **2302** and another terminate request **2428** to the device of the user **1006**.

(274) Therefore, the LP executer **902** never gets access to the user credentials, and the executing code never has access to the credentials. The trusted server **2302** is the one that has access to the user credentials. This way, users can share code while protecting their credentials.

(275) FIG. **25** is a flowchart of a method for managing access to external resources while executing code in a safe environment, according to some example embodiments. While the various operations in this flowchart are presented and described sequentially, one of ordinary skill will appreciate that some or all of the operations may be executed in a different order, be combined or omitted, or be executed in parallel.

(276) Operation **2502** is for receiving, by a controller, a request to execute code.

(277) From operation **2502**, the method **2500** flows to operation **2504** for sending, by the controller to a trusted server, an identifier (ID) associated with the execution of the code and credentials for accessing an external resource.

(278) From operation **2504**, the method **2500** flows to operation **2506** for sending, by the controller to a code executer, the request to execute the code with the ID.

(279) From operation **2506**, the method **2500** flows to operation **2508** to execute the code by the code executer.

(280) After operation **2508**, while executing the code, operations **2510**, **2512**, **2514**, and **2516** are performed. At operation **2510**, an operation for accessing the external resource is detected.

(281) From operation **2510**, the method **2500** flows to operation **2512** where, in response to the detection, a request to access the external resource is sent from the code executer to the trusted server.

(282) From operation **2512**, the method **2500** flows to operation **2514** for accessing, by the trusted server, the external resource using the credentials.

(283) From operation **2514**, the method **2500** flows to operation **2516** for sending a response, from

the trusted server to the code executor, with results from accessing the external resource. The code executor does not have access to the credentials for accessing the external resource.

(284) From operation **2516**, the method **2500** flows to operation **2518**, performed after the code terminates execution to cause presentation on a device of results from the execution of the code.

(285) In one example, the method **2500** further comprises, in response to receiving the request to execute code, generating a sandbox identifier associated with the request and utilizing the sandbox identifier while executing the code by the controller, the trusted server, and the code executor.

(286) In one example, the method **2500** further comprises, in response to receiving the request to execute code, presenting a request for the credentials on a device of a user, and obtaining a token associated with the credentials, the token being used to authenticate a user to obtain access to the external resource.

(287) In one example, the method **2500** further comprises, after execution of the code, saving the code on a task marketplace as a first task without saving the credentials, wherein other users may execute the code of the first task using different credentials.

(288) In one example, the method **2500** further comprises detecting another request from a different user to execute the first task, obtaining credentials for the different user, and executing the code for the first task utilizing the credentials of the different user.

(289) In one example, the method **2500** further comprises combining the first task with a second task created by a second user to create a third task, wherein the third task includes the code from the first task, the code from the second task, and a set of credentials required to execute the third task, the set of credentials required including the credentials required for the first task and the credentials required for the second task.

(290) In one example, the method **2500** further comprises sending status updates, from the code executor to the controller, during the execution of the code for presentation on a computer device.

(291) In one example, the external resource is a file and the request to access the external resource is to read from the file, wherein the credentials correspond to the credentials of a user for accessing a file system where the file is stored.

(292) In one example, the controller manages a database of identifiers associated with code being executed in respective sandboxes.

(293) In one example, the trusted server manages a database of encrypted tokens associated with the credentials of users.

(294) In view of the disclosure above, various examples are set forth below. It should be noted that one or more features of an example, taken in isolation or combination, should be considered within the disclosure of this application.

(295) FIG. **26** is a block diagram illustrating an example of a machine **2600** upon or by which one or more example process embodiments described herein may be implemented or controlled. In alternative embodiments, the machine **2600** may operate as a standalone device or may be connected (e.g., networked) to other machines. In a networked deployment, the machine **2600** may operate in the capacity of a server machine, a client machine, or both in server-client network environments. In an example, the machine **2600** may act as a peer machine in a peer-to-peer (P2P) (or other distributed) network environment. Further, while only a single machine **2600** is illustrated, the term “machine” shall also be taken to include any collection of machines that individually or jointly execute a set (or multiple sets) of instructions to perform any one or more of the methodologies discussed herein, such as via cloud computing, software as a service (SaaS), or other computer cluster configurations.

(296) Examples, as described herein, may include, or may operate by, logic, various components, or mechanisms. Circuitry is a collection of circuits implemented in tangible entities, including hardware (e.g., simple circuits, gates, logic). Circuitry membership may be flexible over time and underlying hardware variability. Circuitries include members that may, alone or in combination, perform specified operations when operating. In an example, the hardware of the circuitry may be

immutably designed to carry out a specific operation (e.g., hardwired). In an example, the hardware of the circuitry may include variably connected physical components (e.g., execution units, transistors, simple circuits), including a computer-readable medium physically modified (e.g., magnetically, electrically, by moveable placement of invariant massed particles) to encode instructions of the specific operation. In connecting the physical components, the underlying electrical properties of a hardware constituent are changed (for example, from an insulator to a conductor or vice versa). The instructions enable embedded hardware (e.g., the execution units or a loading mechanism) to create members of the circuitry in hardware via the variable connections to carry out portions of the specific operation when in operation. Accordingly, the computer-readable medium is communicatively coupled to the other circuitry components when the device is operating. In an example, any of the physical components may be used in more than one member of more than one circuitry. For example, under operation, execution units may be used in a first circuit of a first circuitry at one point in time and reused by a second circuit in the first circuitry or by a third circuit in a second circuitry, at a different time.

(297) The machine **2600** (e.g., computer system) may include a hardware processor **2602** (e.g., a central processing unit (CPU), a hardware processor core, or any combination thereof), a graphics processing unit (GPU **2603**), a main memory **2604**, and a static memory **2606**, some or all of which may communicate with each other via an interlink **2608** (e.g., bus). The machine **2600** may further include a display device **2610**, an alphanumeric input device **2612** (e.g., a keyboard), and a user interface (UI) navigation device **2614** (e.g., a mouse). In an example, the display device **2610**, alphanumeric input device **2612**, and UI navigation device **2614** may be a touch screen display. The machine **2600** may additionally include a mass storage device **2616** (e.g., drive unit), a signal generation device **2618** (e.g., a speaker), a network interface device **2620**, and one or more sensors **2621**, such as a Global Positioning System (GPS) sensor, compass, accelerometer, or another sensor. The machine **2600** may include an output controller **2628**, such as a serial (e.g., universal serial bus (USB)), parallel, or other wired or wireless (e.g., infrared (IR), near field communication (NFC)) connection to communicate with or control one or more peripheral devices (e.g., a printer, card reader).

(298) The mass storage device **2616** may include a machine-readable medium **2622** on which is stored one or more sets of data structures or instructions **2624** (e.g., software) embodying or utilized by any one or more of the techniques or functions described herein. The instructions **2624** may also reside, completely or at least partially, within the main memory **2604**, within the static memory **2606**, within the hardware processor **2602**, or the GPU **2603** during execution thereof by the machine **2600**. In an example, one or any combination of the hardware processor **2602**, the GPU **2603**, the main memory **2604**, the static memory **2606**, or the mass storage device **2616** may constitute machine-readable media.

(299) While the machine-readable medium **2622** is illustrated as a single medium, the term “machine-readable medium” may include a single medium or multiple media (e.g., a centralized or distributed database and/or associated caches and servers) configured to store the one or more instructions **2624**.

(300) The term “machine-readable medium” may include any medium that is capable of storing, encoding, or carrying instructions **2624** for execution by the machine **2600** and that causes the machine **2600** to perform any one or more of the techniques of the present disclosure or that is capable of storing, encoding, or carrying data structures used by or associated with such instructions **2624**. Non-limiting machine-readable medium examples may include solid-state memories and optical and magnetic media. In an example, a massed machine-readable medium comprises a machine-readable medium **2622** with a plurality of particles having invariant (e.g., rest) mass. Accordingly, massed machine-readable media are not transitory propagating signals. Specific examples of massed machine-readable media may include non-volatile memory, such as semiconductor memory devices (e.g., Electrically Programmable Read-Only Memory (EPROM),

Electrically Erasable Programmable Read-Only Memory (EEPROM)) and flash memory devices; magnetic disks, such as internal hard disks and removable disks; magneto-optical disks; and CD-ROM and DVD-ROM disks.

(301) The instructions **2624** may further be transmitted or received over a communications network **2626** using a transmission medium via the network interface device **2620**.

(302) Throughout this specification, plural instances may implement components, operations, or structures described as a single instance. Although individual operations of one or more methods are illustrated and described as separate operations, one or more of the individual operations may be performed concurrently, and nothing requires that the operations be performed in the order illustrated. Structures and functionality presented as separate components in example configurations may be implemented as a combined structure or component. Similarly, structures and functionality presented as a single component may be implemented separately. These and other variations, modifications, additions, and improvements fall within the scope of the subject matter herein.

(303) The embodiments illustrated herein are described in sufficient detail to enable those skilled in the art to practice the teachings disclosed. Other embodiments may be used and derived therefrom, such that structural and logical substitutions and changes may be made without departing from the scope of this disclosure. The Detailed Description, therefore, is not to be taken in a limiting sense, and the scope of various embodiments is defined only by the appended claims, along with the full range of equivalents to which such claims are entitled.

(304) Additionally, as used in this disclosure, phrases of the form “at least one of an A, a B, or a C,” “at least one of A, B, and C,” and the like should be interpreted to select at least one from the group that comprises “A, B, and C.” Unless explicitly stated otherwise in connection with a particular instance, in this disclosure, this manner of phrasing does not mean “at least one of A, at least one of B, and at least one of C.” As used in this disclosure, the example “at least one of an A, a B, or a C” would cover any of the following selections: {A}, {B}, {C}, {A, B}, {A, C}, {B, C}, and {A, B, C}.

(305) Moreover, plural instances may be provided for resources, operations, or structures described herein as a single instance. Additionally, boundaries between various resources, operations, modules, engines, and data stores are somewhat arbitrary, and particular operations are illustrated in the context of specific illustrative configurations. Other allocations of functionality are envisioned and may fall within the scope of various embodiments of the present disclosure. In general, structures and functionality presented as separate resources in the example configurations may be implemented as a combined structure or resource. Similarly, structures and functionality presented as a single resource may be implemented as separate resources. These and other variations, modifications, additions, and improvements fall within a scope of embodiments of the present disclosure as represented by the appended claims. Accordingly, the specification and drawings are to be regarded in an illustrative rather than a restrictive sense.

Claims

1. A computer-implemented method comprising: receiving, by a controller, a request to execute code; sending, by the controller to a trusted server, an identifier (ID) associated with the execution of the code and credentials for accessing an external resource; sending, by the controller to a code executor, the request to execute the code with the ID; executing the code by the code executor; while executing the code, perform operations comprising: detecting an operation for accessing the external resource; in response to the detecting, sending, from the code executor to the trusted server, a request to access the external resource; accessing, by the trusted server, the external resource using the credentials; and sending a response, from the trusted server to the code executor, with results from accessing the external resource, wherein the code executor does not have access

to the credentials for accessing the external resource; and after the code terminates execution, causing presentation on a device of results from the execution of the code.

2. The method as recited in claim 1, further comprising: in response to receiving the request to execute code, generating a sandbox identifier associated with the request; and utilizing the sandbox identifier while executing the code by the controller, the trusted server, and the code executor.
3. The method as recited in claim 1, further comprising: in response to receiving the request to execute code, presenting a request for the credentials on a device of a user; and obtaining a token associated with the credentials, the token being used to authenticate a user to obtain access to the external resource.
4. The method as recited in claim 1, further comprising: after execution of the code, saving the code on a task marketplace as a first task without saving the credentials, wherein other users may execute the code of the first task using different credentials.
5. The method as recited in claim 4, further comprising: detecting another request from a different user to execute the first task; obtaining credentials for the different user; and executing the code for the first task utilizing the credentials of the different user.
6. The method as recited in claim 4, further comprising: combining the first task with a second task created by a second user to create a third task, wherein the third task includes the code from the first task, the code from the second task, and a set of credentials required to execute the third task, the set of credentials required including the credentials required for the first task and the credentials required for the second task.
7. The method as recited in claim 1, further comprising: sending status updates, from the code executor to the controller, during the execution of the code for presentation on a computer device.
8. The method as recited in claim 1, wherein the external resource is a file and the request to access the external resource is to read from the file, wherein the credentials correspond to the credentials of a user for accessing a file system where the file is stored.
9. The method as recited in claim 1, wherein the controller manages a database of identifiers associated with code being executed in respective sandboxes.
10. The method as recited in claim 1, wherein the trusted server manages a database of encrypted tokens associated with credentials of users.
11. A system comprising: a memory comprising instructions; and one or more computer processors, wherein the instructions, when executed by the one or more computer processors, cause the system to perform operations comprising: receiving, by a controller, a request to execute code; sending, by the controller to a trusted server, an identifier (ID) associated with the execution of the code and credentials for accessing an external resource; sending, by the controller to a code executor, the request to execute the code with the ID; executing the code by the code executor; while executing the code, perform operations comprising: detecting an operation for accessing the external resource; in response to the detecting, sending, from the code executor to the trusted server, a request to access the external resource; accessing, by the trusted server, the external resource using the credentials; and sending a response, from the trusted server to the code executor, with results from accessing the external resource, wherein the code executor does not have access to the credentials for accessing the external resource; and after the code terminates execution, causing presentation on a device of results from the execution of the code.
12. The system as recited in claim 11, wherein the instructions further cause the one or more computer processors to perform operations comprising: in response to receiving the request to execute code, generating a sandbox identifier associated with the request; and utilizing the sandbox identifier while executing the code by the controller, the trusted server, and the code executor.
13. The system as recited in claim 11, wherein the instructions further cause the one or more computer processors to perform operations comprising: in response to receiving the request to execute code, presenting a request for the credentials on a device of a user; and obtaining a token associated with the credentials, the token being used to authenticate a user to obtain access to the

external resource.

14. The system as recited in claim 11, wherein the instructions further cause the one or more computer processors to perform operations comprising: after execution of the code, saving the code on a task marketplace as a first task without saving the credentials, wherein other users may execute the code of the first task using different credentials.

15. The system as recited in claim 14, wherein the instructions further cause the one or more computer processors to perform operations comprising: detecting another request from a different user to execute the first task; obtaining credentials for the different user; and executing the code for the first task utilizing the credentials of the different user.

16. A non-transitory machine-readable storage medium including instructions that, when executed by a machine, cause the machine to perform operations comprising: receiving, by a controller, a request to execute code; sending, by the controller to a trusted server, an identifier (ID) associated with the execution of the code and credentials for accessing an external resource; sending, by the controller to a code executor, the request to execute the code with the ID; executing the code by the code executor; while executing the code, perform operations comprising: detecting an operation for accessing the external resource; in response to the detecting, sending, from the code executor to the trusted server, a request to access the external resource; accessing, by the trusted server, the external resource using the credentials; and sending a response, from the trusted server to the code executor, with results from accessing the external resource, wherein the code executor does not have access to the credentials for accessing the external resource; and after the code terminates execution, causing presentation on a device of results from the execution of the code.

17. The non-transitory machine-readable storage medium as recited in claim 16, wherein the machine further performs operations comprising: in response to receiving the request to execute code, generating a sandbox identifier associated with the request; and utilizing the sandbox identifier while executing the code by the controller, the trusted server, and the code executor.

18. The non-transitory machine-readable storage medium as recited in claim 16, wherein the machine further performs operations comprising: in response to receiving the request to execute code, presenting a request for the credentials on a device of a user; and obtaining a token associated with the credentials, the token being used to authenticate a user to obtain access to the external resource.

19. The non-transitory machine-readable storage medium as recited in claim 16, wherein the machine further performs operations comprising: after execution of the code, saving the code on a task marketplace as a first task without saving the credentials, wherein other users may execute the code of the first task using different credentials.

20. The non-transitory machine-readable storage medium as recited in claim 19, wherein the machine further performs operations comprising: detecting another request from a different user to execute the first task; obtaining credentials for the different user; and executing the code for the first task utilizing the credentials of the different user.
